AIORI NATIONAL HACKATHON 3

A3-PS027-TC313: IPv6 Website & Dependency Readiness Auditor

Detecting Client-Side Failures Caused by Legacy IPv4 Dependencies on Pure-IPv6 Networks

Presenter: Saurabh Gangwar

Contact: saurabh.bcs2025552@invertisuniversity.ac.in

Date: October 5, 2026

Institution: Invertis University, Computer Science & Engineering

Track: IPv6 Deployment & Tooling

Project Code: A3-PS027-TC313 (V6Lighthouse)

Problem Description & Operational Failure Modes

- **The Partial Migration Trap:** Enterprises configure AAAA DNS records on their apex servers, assuming 100% IPv6 readiness. However, modern web frontends link dozens of external third-party assets.
- **Unverified 3rd-Party Dependencies:** Websites depend on external CDNs, fonts, analytics, metrics, and payment SDKs (e.g., Stripe, jQuery, Google Fonts) hosted on unvetted, legacy IPv4-only domains.
- **Silent Pure-IPv6 Network Failures:** Clients on pure-IPv6 cellular networks or corporate subnets fail to resolve or route to IPv4 assets. This causes missing UI styling, dropped checkouts, and stalled JavaScript scripts without triggering server-side alerts.
- **Diagnostic Tooling Vacuum:** Standard tools (Ping, Dig, Google Lighthouse) evaluate only the apex hostname, offering zero visibility into the third-party dependency graph during runtime.

Background: Transition Realities & The Dependency Gap

- **Global IPv6 Transition:** IPv4 address exhaustion has driven mobile operators to deploy pure-IPv6 access networks utilizing transition mechanisms (NAT64/DNS64).
- **The Dual-Stack Fallacy:** Websites believe dual-stack apex deployment solves compatibility. In reality, client browsers execute external script tags that bypass Happy Eyeballs fallback algorithms entirely when the resource domain lacks IPv6 addresses.
- **Proposed Optimization:** The V6Lighthouse engine bridges this gap by spawning headless browser sandboxes to harvest 100% of runtime outgoing HTTP requests and checking each domain's IPv6 reachability via parallel low-level DNS threads.

Literature Survey & Standard Specifications

- **RFC 8200 (Internet Protocol, Version 6):** Defines the 128-bit address architecture, extension header processing, and mandatory DNS AAAA record mapping for internet hosts.
- **RFC 6555 (Happy Eyeballs v2):** Specifies dual-stack connection racing to reduce user-visible delays. Documents how pure-IPv6 client environments fail when third-party servers provide no AAAA endpoints.
- **RFC 6883 (IPv6 Guidance for Web & App Providers):** Explicitly warns that partial dual-stack transitions fail when dependent assets are hosted on legacy infrastructure.
- **W3C Resource Timing Standard:** Establishes performance measurement guidelines and request interfaces, enabling precise runtime tracking of third-party domains.

Proposed Solution: The V6Lighthouse Platform

- **1. Target Ingestion:** Developer inputs a target hostname. Express orchestrator sanitizes the URL and initializes the job.
- **2. Apex DNS Audit:** Parallel native Node.js resolver validates primary server infrastructure by checking AAAA, NS, and MX records.
- **3. Headless Harvester:** Spawns a sandboxed Puppeteer browser container, intercepts outgoing HTTP requests, isolates external origins, and tags MIME types.
- **4. Scoring & Remediation:** Computes a weighted Readiness Score (0-100%) and generates context-aware migration recommendations (e.g., cdnjs or local hosting).

Technical Comparison Matrix

Dimension	Traditional Tools (e.g., Lighthouse, Dig)	V6Lighthouse Auditor
Audit Scope	Apex domain only (AAAA record lookup).	100% of runtime external dependencies (JS, CSS, fonts, APIs).
Harvesting Method	Static HTML regex parsing.	Dynamic headless Puppeteer network request interception.
DNS Accuracy	Prone to browser-internal DNS cache pollution.	Parallel native c-ares low-level asynchronous resolver (dns.promises).
Output & Guidance	Binary Pass/Fail indicator.	Lighthouse-style weighted score, targeted CDN remediation fixes, and developer CSV export.
CI/CD Gating	Manual web console testing.	Headless CLI with JSON exit codes for GitHub Actions pipelines (--fail-under 80).

Decoupled Subsystem Architecture (UML Model)

- **Presentation Layer (React 18 SPA):** Handles target URL intake, circular SVG metrics dashboard, searchable dependency ledger, and CSV export.
- **Communication Bridge:** Secure REST API (POST /api/audit on Port 5000).
- **Orchestrator Tier (Express.js + TypeScript):** Handles schema validation, asynchronous job queuing, and scoring calculations.
- **Parallel Worker Tier:**
  - **Headless Harvester (Puppeteer):** Sandboxed Chromium instance executing DOM-rendering and dynamic network interception.
  - **Native DNS Engine (dns/promises):** Direct c-ares socket lookups running in a non-blocking thread pool.

End-to-End Data Flow Pipeline

- **1. URL Input:** Dashboard triggers POST query -> Sanitized by Express controller.
- **2. Apex DNS Thread:** Dispatches parallel lookup jobs for AAAA, NS, and MX records.
- **3. Dynamic Rendering:** Puppeteer opens context -> intercepts dynamic outgoing HTTP/S calls.
- **4. Deduplication Gate:** Deduplicates 200+ network calls into unique third-party domains.
- **5. Parallel DNS Check:** Batches unique origins into resolve6() threads.
- **6. Score Engine:** Computes formula: **$Score = S_{Host} * 50 + (D_{Compliant} / D_{Total}) * 50$** and serializes JSON payload to client UI.

Engineering Milestones & Deployment Schedule

- **Phase 1 (Week 1): Core Engine & DNS:** Establish Express TypeScript monorepo, write Puppeteer interception scripts, configure c-ares async resolver. **(Status: 100% Complete)**
- **Phase 2 (Week 2): UI & Scoring:** Design React 18 dashboard, configure SVG circular charts, build data tables, and implement client CSV exporter. **(Status: 100% Complete)**
- **Phase 3 (Week 3): Advanced Features:** Build simulated pure-IPv6 sandbox routing tests and implement automated CDN code rewriter. **(Status: Hackathon Ready)**
- **Phase 4 (Week 4): CI/CD & Scale:** Package CLI tool for GitHub Actions gating (--fail-under 80) and deploy multi-region distributed nodes. **(Status: Roadmap)**

Standards, Specifications & References Cited

1. **S. Deering, R. Hinden.** "Internet Protocol, Version 6 (IPv6) Specification." IETF RFC 8200 (Internet Standard), July 2017.
2. **D. Wing, A. Yourtchenko.** "Happy Eyeballs: Success with Dual-Stack Hosts." IETF RFC 6555, Proposed Standard, April 2012.
3. **M. Blanchet.** "IPv6 Guidance for Internet Content Providers and Application Service Providers." IETF RFC 6883, Informational, March 2013.
4. **W3C Web Performance Working Group.** "Resource Timing Level 2." W3C Recommendation, 2019.
5. **Chrome DevTools Team.** "Puppeteer: Headless Chrome Node.js API for Network Interception." Google Open Source Documentation, 2024.

Thank You for Your Attention!

- **Problem Statement:** PS-027 (A3-PS027-TC313)
- **Project Name:** V6Lighthouse Auditor
- **Presenter:** Saurabh Gangwar (Computer Science & Engineering, Invertis University)
- **Contact Email:** saurabh.bcs2025552@invertisuniversity.ac.in
- **Open for Discussion:** Live demonstration of real-time dependency harvesting, parallel DNS resolution benchmarks, and automated CDN remediation.